1. What is NodeJS?

NodeJS is an open-source, cross-platform JavaScript runtime environment engine used for executing JavaScript code outside the browser. It is built on Google Chrome's V8 JavaScript engine. Some of the key features of NodeJS are mentioned below:

- Single-threaded
- Non-Blocking, Asynchronous I/O
- Cross-platform
- Fast Execution V8 Engine
- Real-Time Data Handling

2. What is NPM?

NPM stands for the Node Package Manager. It is the package manager for the NodeJS environment. It is used to install, share, and manage dependencies (libraries, tools, or packages) in JavaScript applications. Below are the following key points about the NPM:

- NPM uses a package.json file in NodeJS projects to track project dependencies, versions, scripts, and metadata like the project's name and version.
- NPM is accessed by a command-line interface (CLI). Common commands include npm install to install packages, npm update to update them, and npm uninstall to remove them.

3. What is piping in NodeJS?

In NodeJS, **piping** refers to the process of passing the output of one stream directly into another stream. It allows data to flow through multiple streams without needing to store it in memory or temporarily write it to disk. This is a common pattern used in file handling, HTTP requests, and other I/O operations in NodeJS.

4. What is the difference between Synchronous and Asynchronous functions?

Synchronous Functions	Asynchronous Functions
Blocks the execution until the task completes.	Does not block the execution; allows other tasks to proceed concurrently.
Executes tasks sequentially; each task must be completed before the next one starts.	Initiate tasks and proceed with other operations while waiting for completion.
Returns the result immediately after completion.	Typically returns a promise or callback or uses event handling to handle the result upon completion.
Errors can be easily caught with try-catch blocks.	Error handling is more complex and often involves callbacks, promises, or async/await syntax.
Suitable for simple, sequential tasks with predictable execution flow.	Ideal for I/O-bound operations, network requests, and tasks requiring parallel processing.

JavaScript is syncronous but help of libuv make nodejs asyncronous

5. Why is NodeJS single-threaded?

NodeJS is single-threaded because it's based on the asynchronous, non-blocking nature of JavaScript. This design makes it simpler to develop and maintain, and it allows NodeJS to handle many concurrent requests efficiently.

6. If NodeJS is single-threaded, then how does it handle concurrency?

NodeJS is single-threaded, but it can handle concurrency efficiently through its event-driven, non-blocking I/O model. While the event loop in NodeJS runs on a single thread, it doesn't block the execution of other tasks when waiting for I/O operations, such as file reads or database queries. Instead, NodeJS delegates these I/O tasks to the system's kernel, allowing it to continue processing other requests. Once the I/O operation is complete, the corresponding callback is added to a queue and processed by the event loop. This non-blocking approach enables NodeJS to handle multiple concurrent tasks without waiting for each one to finish sequentially.

"What is Single Thread in Node.js?"

say:

> **Node.js is single-threaded, which means it uses only one main thread to execute JavaScript code. This thread is responsible for handling all incoming requests and executing them one by one through the event loop.**

7. What are the module in NodeJS?

In a NodeJS Application, a Module can be considered as a block of code that provides a simple or complex functionality that can communicate with external application. Modules can be organized in a single file or a collection of multiple files/folders. They are useful because of their reusability and ability to reduce the complexity of code into smaller pieces. **Examples of modules are.** http, fs, os, path, etc.

Feature	CommonJS	ES Modules
Import	`require()`	`import`
Export	`module.exports`	`export`
Loading	Synchronous	Asynchronous
Usage	Node (old)	Node + Browser
Flexibility	Dynamic import	Static (top-level)

***CommonJS vs ES Module (import / export)**

Node.js has two module systems: CommonJS which uses require and module.exports, and ES Modules which use import and export. ES Modules are the modern standard, while CommonJS is the older Node.js system.

8. What is the purpose of the 'require' keyword in NodeJS?

The require keyword in NodeJS is used to include and import modules (external or built-in) into a NodeJS application.

Example

```
const http = require('http')   //imports the HTTP module to create a server.
```

9. What is middleware?

Middleware is the function that works between the request and the response cycle. Middleware gets executed after the server receives the request and before the controller sends the response.

10. What is the role of environment variables in Node.js development?

Environment variables in Node.js are dynamic variables used to configure and customize application behavior based on the environment. They provide a convenient way to store sensitive information like database credentials or API keys without hardcoding them into the codebase.

Example: Accessing Environment Variables

```
const port = process.env.PORT || 3000;
console.log(`Server is running on port ${port}`);
```

11. What do you mean by event loop in NodeJS?

The event loop in Node.js is a mechanism that allows asynchronous tasks to be handled efficiently without blocking the execution of other operations. It:

- Executes JavaScript synchronously first and then processes asynchronous operations.
- Delegates heavy tasks like I/O operations, timers, and network requests to the libuv library.
- Ensures smooth execution of multiple operations by queuing and scheduling callbacks efficiently.

  libuv = “the library/engine that makes async possible in Node.js”

  The event loop is like a manager. When a request comes, if it’s a heavy task like database or file system, it assigns it to workers (libuv). Once the work is done, the result comes back and the event loop executes the callback

  “Full Execution Priority ”

  1. Synchronous code (Call Stack)
  2. process.nextTick() ← highest priority ⚡
  3. Promise (microtasks)
  4. Timers (setTimeout, setInterval)
  5. Poll (I/O callbacks)
  6. Check (setImmediate)
  7. Close callbacks

Why Event Loop is important?

The Event Loop is essential in Node.js because it allows non-blocking, asynchronous operations to be handled efficiently, even though Node.js operates on a single thread.

- Enables non-blocking execution despite Node.js being single-threaded.
- Helps handle I/O-bound tasks efficiently.
- Makes Node.js suitable for scalable applications like web servers.

Thread Pool? > Thread pool in Node.js is a group of worker threads managed by libuv that handle heavy operations like file system and cryptographic tasks. It allows Node.js to remain non-blocking by offloading these tasks from the main thread.

12. What is package.json in NodeJS?

The **package.json** file is the heart of Node.js system. It is the manifest file of any Node.js project and contains the metadata of the project. The package.json file is the essential part to understand, learn and work with the Node.js. It is the first step to learn about development in Node.js.

13. What are promises in NodeJS?

A promise is an advancement of callbacks in NodeJS. In other words, a promise is a JavaScript object that is used to handle all the asynchronous data operations. While developing an application, you may encounter that you are using a lot of nested callback functions, which causes a problem of callback hell. Promises solve this problem of callback hell.

JWT(Jsonwebtoken)?

A JSON Web Token (JWT) is a secure way to send information between a client and a server. It is mainly used in web applications and APIs to verify users and prevent unauthorized access. A JWT is JSON data secured with a cryptographic signature.

JSON Web Tokens consist of three parts separated by dots (**.**), which are:

1. Header: Contains metadata about the token, such as the algorithm used for signing.

2. Payload: Stores the claims, i.e., data being transmitted.

3. Signature: Ensures the token's integrity and authenticity.

. What is the difference between Authentication and Authorization?

Authentication and Authorization are both security processes, but they serve different purposes.

- **Authentication** is about verifying the user's identity. It answers *"Who is the user?"*
  In real projects, I've implemented authentication using **JWT tokens**, where users log in with email/password and receive a token after successful verification.
- **Authorization** is about checking what the authenticated user is allowed to do. It answers *"What actions can the user perform?"*
  For example, in a role-based system, only admins can delete data, while normal users can only view or update their own data.

"In my project, I used JWT for authentication and created middleware to check user roles for authorization before allowing access to protected routes."

Flow in real application:

1. User logs in → backend verifies credentials (**Authentication**)
2. Backend sends JWT token
3. On each request, token is verified
4. Based on user role → access is granted or denied (**Authorization**)

What is CORS?

CORS (Cross-Origin Resource Sharing) is a security feature that restricts requests between different domains. In Node.js, we use the `cors` middleware to allow cross-origin requests.

What is REST API? What are HTTP status codes?

REST API is a way to build APIs using HTTP methods like GET, POST, PUT, DELETE. Common status codes include 200 (success), 201 (created), 400 (bad request), 401 (unauthorized), 404 (not found), and 500 (server error).

How does `require()` work internally?

When we use `require()` in Node.js, it loads the module, executes it, and caches it. If the same module is required again, Node.js returns the cached version instead of re-executing it, which improves performance.

Difference between `process.nextTick()` and `setImmediate()`

> `process.nextTick()` executes immediately after the current operation, before the event loop continues, while `setImmediate()` runs in the check phase of the event loop. So, `process.nextTick()` has higher priority than `setImmediate()`.

What is middleware in Express (real usage)?

> Middleware functions are used to process requests before sending a response. In my project, I used middleware for authentication, where I verified JWT tokens before allowing access to protected routes.

What is caching?

Caching involves storing copies of data in a temporary storage location, or cache, to quickly access it upon subsequent requests. This approach minimizes the need to repeatedly retrieve data from a slower backend data store, such as a database or an external API.

What is MVC architecture?

MVC (Model-View-Controller) is a software architecture pattern used to organize code into three separate components: Model, View, and Controller.

It helps in separating concerns, making the application more scalable and maintainable.

What is clustering in Node.js?

Clustering in Node refers to a technique used to enhance the performance and scalability of NodeJS applications by utilizing the capabilities of multi-core systems. With clustering, you can create multiple instances of the NodeJS process, known as workers, each running on a separate CPU core. By distributing the workload across these workers, applications can handle a higher volume of requests and utilize the available system resources more efficiently.

What is stream in Node.js?

Streams in Node.js are used to handle data efficiently by processing it in small chunks instead of loading the entire data into memory at once.

System design

What is load balancing?

Load balancing is a technique used to distribute incoming network traffic across multiple servers to ensure no single server gets overloaded, improving performance and reliability.

What is API Gateway?

"API Gateway is a centralized entry point that receives client requests, performs tasks such as authentication, rate limiting, logging, and request routing, then forwards the request to the appropriate backend service. It simplifies client communication and is commonly used in microservices architectures."

DataBase MongoDB

1. What is Mongodb?

MongoDB is a NoSQL, document-oriented database that stores data in BSON (Binary JSON) documents instead of tables and rows like traditional SQL databases. It has a flexible schema, which means documents in the same collection can have different fields without requiring schema changes. MongoDB is designed for high performance, scalability, and handling large amounts of semi-structured or unstructured data. It is commonly used with JavaScript-based stacks like MERN (MongoDB, Express.js, React, Node.js).

2.Embedding vs Referencing (MongoDB) ?

Embedding	Referencing
Store related data inside the same document.	Store the related document's _id and keep data in another collection.
Faster reads, no join needed.	Less duplication, easier updates.
Best for small, tightly related data.	Best for large, reusable, or frequently updated data.

Embedding	Referencing

1. Embedding example

User document with address embedded

Why use it? A user's address is usually accessed together with the user profile, so one query fetches everything.

2. Referencing example

Users collection

Posts collection

Here, userId references the user document. In Mongoose, you can fetch related data with populate().

Real-world examples

Use Embedding ✅	Use Referencing ✅
User → Address	User → Posts
User → Preferences	Order → Product
Order → Shipping address snapshot	Blog Post → Author
Small fixed-size arrays	Comments that may grow indefinitely

One-line

Embedding stores related data inside the same document and is best for small, tightly coupled data that is read together. Referencing stores the related document's _id in another collection and is best for large, shared, or frequently updated data because it avoids duplication and scales better.

Quick follow-up they may ask: “For an e-commerce app, would you embed products inside orders?” → Usually yes, as a snapshot (name, price, quantity at purchase time), while the master product remains in a separate products collection and is referenced elsewhere.

3. SQL vs MongoDb?

SQL is a relational database that stores data in tables with a fixed schema and uses SQL for querying. It's ideal for applications requiring complex relationships, joins, and strong ACID transactions, such as banking or financial systems.

MongoDB is a NoSQL document database that stores data in BSON documents with a flexible schema. It is well-suited for applications where data structures evolve frequently, such as e-commerce, content management systems, and social media platforms. It also scales horizontally using sharding.

The choice depends on the application's requirements. If data consistency and complex relationships are the priority, I would choose SQL. If flexibility, rapid development, and horizontal scalability are more important, I would choose MongoDB.

4.What is Aggregation in MongoDB?

> **Aggregation in MongoDB is a framework used to process and analyze data by passing documents through multiple stages called an aggregation pipeline. Each stage transforms or filters the data and passes the result to the next stage. It is used for operations like filtering, grouping, sorting, calculating totals, averages, counts, and joining collections. It's similar to SQL operations like `GROUP BY`, `SUM`, and `COUNT`.**

Example

Suppose you have an `orders` collection.

```
[
  {
    "customer": "Manish",
    "amount": 500
  },
  {
    "customer": "Rahul",
    "amount": 1000
  },
  {
    "customer": "Manish",
    "amount": 700
  }
]
```

Find the total amount spent by each customer

```
db.orders.aggregate([
  {
    $group: {
      _id: "$customer",
      totalAmount: { $sum: "$amount" }
    }
  }
])
```

Output

```
[
  {
    "_id": "Manish",
    "totalAmount": 1200
  },
  {
    "_id": "Rahul",
    "totalAmount": 1000
  }
]
```

Common Aggregation Pipeline Stages

Stage	Purpose
`$match`	Filter documents (similar to `WHERE`)
`$group`	Group documents (`GROUP BY`)
`$sort`	Sort results
`$project`	Select or transform fields
`$limit`	Limit the number of documents
`$skip`	Skip documents (pagination)
`$count`	Count documents
`$lookup`	Join another collection
`$unwind`	Split array elements into separate documents

Example Pipeline

```
db.orders.aggregate([
  { $match: { status: "Completed" } },
  { $group: { _id: "$customer", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } }
])
```

This pipeline:

1. Filters only completed orders.
2. Groups them by customer.
3. Calculates each customer's total spending.
4. Sorts customers by total spending in descending order.

SQL

```
SELECT customer, SUM(amount)
FROM orders
GROUP BY customer;
```

MongoDB

```
db.orders.aggregate([
  {
    $group: {
      _id: "$customer",
      total: { $sum: "$amount" }
    }
  }
])
```

Difference between `find()` and `aggregate()`

`find()`

- Retrieves documents.
- Used for simple filtering.
- Faster for basic queries.

Example:

```
db.users.find({ age: { $gt: 18 } })
```

`aggregate()`

- Processes data through multiple stages.
- Used for filtering, grouping, calculations, sorting, and joins.
- More powerful for analytics and reporting.

Example:

```
db.orders.aggregate([
  {
    $group: {
      _id: "$customer",
      total: { $sum: "$amount" }
    }
  }
])
```

5.What is Indexing in MongoDB?

> **Indexing is a data structure that MongoDB uses to improve the speed of data retrieval. Without an index, MongoDB has to scan every document in a collection to find matching data (collection scan). With an index, MongoDB can quickly locate the required documents, making queries much faster. However, indexes consume additional storage and slightly slow down insert, update, and delete operations because the indexes must also be maintained.**

Example

Suppose you have a `users` collection with **1 million documents**.

You search by email:

```
db.users.find({ email: "manish@gmail.com" })
```

Without an Index

MongoDB checks every document one by one until it finds the match.

```
Doc1 ❌
Doc2 ❌
Doc3 ❌
```

```
...
Doc1000000 ✅
```

This is called a **Collection Scan**, which is slow for large datasets.

With an Index

Create an index:

```
db.users.createIndex({ email: 1 })
```

Now MongoDB uses the index to jump directly to the matching document instead of scanning the whole collection.

```
Index
|
|-- abc@gmail.com
|-- manish@gmail.com   ✅
|-- xyz@gmail.com
```

Query performance becomes much faster.

Types of Indexes

1. Single Field Index

```
db.users.createIndex({ email: 1 })
```

Indexes only the `email` field.

---

2. Compound Index

```
db.users.createIndex({
  name: 1,
  age: -1
})
```

Indexes multiple fields together.

---

3. Unique Index

```
db.users.createIndex(
  { email: 1 },
```

```
  { unique: true }
)
```

Prevents duplicate email addresses.

---

4. Text Index

```
db.posts.createIndex({
  title: "text"
})
```

Used for text search.

---

5. TTL (Time-To-Live) Index

Automatically deletes expired documents.

Common use cases:

- OTPs
- Sessions
- Logs

Advantages

- Faster queries
- Better search performance
- Improved sorting performance
- Better filtering

Disadvantages

- Uses extra disk space
- Slows inserts, updates, and deletes because indexes must also be updated
- Too many indexes can reduce overall performance

Q: Why don't we create indexes on every field?

Answer:

> **Because every index consumes storage and increases the cost of write operations. Whenever a document is inserted, updated, or deleted, MongoDB must update the corresponding indexes. Therefore, we create indexes only on fields that are frequently used in searches, filters, sorting, or joins.**

How to View Indexes

```
db.users.getIndexes()
```

Delete an Index

```
db.users.dropIndex({ email: 1 })
```

> **"What happens if there is no index?"**

A strong answer:

> **Without an index, MongoDB performs a collection scan, meaning it checks every document in the collection to find matching data. This becomes increasingly slow as the collection grows, which is why indexing is essential for frequently queried fields.**

6.Q: What is a Document, Collection, and Database in MongoDB?

> **MongoDB organizes data into three levels: Database, Collection, and Document.**

- **Database** is the top-level container that stores one or more collections.
- **Collection** is a group of related documents. It is similar to a table in SQL.
- **Document** is the basic unit of data in MongoDB. It is stored in BSON format and contains data as key-value pairs, similar to a JSON object.

Example

```
Database: EcommerceDB
```

```
Collections:
- Users
- Products
- Orders

Users Collection:
{
  "_id": "1",
  "name": "Manish",
  "email": "manish@gmail.com"
}
```

Here:

- **EcommerceDB** → Database
- **Users** → Collection
- `{ "_id": "1", "name": "Manish", "email": "manish@gmail.com" }` → Document

> **A Database contains Collections, Collections contain Documents, and Documents store the actual data in BSON format as key-value pairs.**

> **Yes. MongoDB is schema-flexible, so documents in the same collection can have different fields. However, in production applications, we usually use Mongoose schemas to maintain consistency and validate the data.**

7.BSON and JSON in MongoDB.

JSON (JavaScript Object Notation) is a lightweight, human-readable format used to represent and exchange data as key-value pairs.

BSON (Binary JSON) is the binary-encoded format that MongoDB uses internally to store documents. BSON extends JSON by supporting additional data types such as `ObjectId`, `Date`, `Binary`, `Decimal128`, and more. It is optimized for efficient storage and faster processing inside MongoDB.

Q: What is Sharding in MongoDB?

> **Sharding is MongoDB's horizontal scaling technique used to distribute large amounts of data across multiple servers (called shards). Instead of storing all data on a single server, MongoDB splits the data into smaller chunks and stores them across different shards. This improves storage capacity, query performance, and allows the database to handle high traffic and large datasets.**

Why do we need Sharding?

Imagine your application has **500 million users**.

If all the data is stored on one server:

- ❌ Storage may run out.
- ❌ Queries become slower.
- ❌ High traffic can overload the server.

Sharding solves this by distributing the data across multiple servers.

Example

Suppose you have **9 million users**.

Without Sharding

```
Server 1
-----------------
User 1
User 2
User 3
...
User 9000000
```

One server stores everything.

With Sharding

```
Shard 1          Shard 2          Shard 3
--------         --------         --------
User 1-3M        User 3M-6M       User 6M-9M
```

Each shard stores only part of the data.

Components of Sharding

1. Shard

- Stores a portion of the data.

2. Config Server

- Stores metadata about where the data is located.

3. Query Router (`mongos`)

- Receives client requests and routes them to the correct shard.

```
Application
      |
   mongos
      |
-------------------------
|          |            |
Shard 1   Shard 2    Shard 3
      |
 Config Servers
```

Advantages

- ✅ Horizontal scaling
- ✅ Better query performance
- ✅ Handles very large datasets
- ✅ Supports high read/write traffic
- ✅ No single server becomes a bottleneck

Disadvantages

- More complex to configure and maintain
- Choosing the wrong shard key can cause uneven data distribution
- Cross-shard operations can be slower

What is a Shard Key?

A **Shard Key** is the field MongoDB uses to decide **which shard stores a document**.

Example:

```
{
  "_id": 1,
  "userId": 1001,
  "name": "Manish"
}
```

If `userId` is the shard key:

- Users `1–1000000` → Shard 1
- Users `1000001–2000000` → Shard 2
- Users `2000001–3000000` → Shard 3

A good shard key should:

- Have high cardinality (many unique values)
- Distribute data evenly
- Avoid creating hotspots

Sharding vs Replication

Sharding	Replication
Distributes data across multiple servers	Copies the same data to multiple servers
Used for scalability	Used for high availability and fault tolerance
Increases storage capacity	Provides backup and failover
Different data on each shard	Same data on each replica

> **Sharding is a horizontal scaling technique in MongoDB that distributes data across multiple servers (shards) to improve performance, storage capacity, and handle large volumes of data and traffic.**

Q: What's the difference between Vertical Scaling and Horizontal Scaling?

Answer:

- **Vertical Scaling (Scale Up):** Increase the CPU, RAM, or storage of a single server.
- **Horizontal Scaling (Scale Out):** Add more servers and distribute the data among them. MongoDB uses **sharding** for horizontal scaling

This is a very common MongoDB interview question.

Q: What is `_id` and ObjectId in MongoDB?

Interview Answer (1 minute)

> **`_id` is a unique identifier for every document in MongoDB. Every document must have an `_id` field, which acts like the Primary Key in SQL. If we don't provide an `_id`, MongoDB automatically generates one using an `ObjectId`.**

Example:

```
{
  "_id": ObjectId("6867a9f3b8d4e5f6a7b8c9d0"),
  "name": "Manish",
  "age": 23
}
```

What is ObjectId?

> **ObjectId is a 12-byte unique identifier generated by MongoDB to uniquely identify each document. It ensures that every document has a unique `_id`.**

Structure of ObjectId

An ObjectId consists of **12 bytes (24 hexadecimal characters).**

`ObjectId("6867a9f3b8d4e5f6a7b8c9d0")`

It contains:

Bytes	Description
4 Bytes	Timestamp (creation time)
5 Bytes	Random value (machine/process identifier)
3 Bytes	Incrementing counter

Breakdown

```
6867a9f3 | b8d4e5f6a7 | b8c9d0
---------|------------|--------
Timestamp | Random     | Counter
```

- **Timestamp** → When the document was created.
- **Random Value** → Helps ensure uniqueness across machines and processes.
- **Counter** → Increments with each generated ObjectId.

Why is ObjectId useful?

- Automatically generates unique IDs.
- No need to create IDs manually.
- Contains the creation timestamp.
- Efficient for indexing and querying.

SQL Comparison

SQL	MongoDB
Primary Key	`_id`
Auto Increment	`ObjectId()`

Example

Insert a document:

```
db.users.insertOne({
    name: "Manish",
    age: 23
});
```

MongoDB automatically creates:

```
{
  "_id": ObjectId("6867a9f3b8d4e5f6a7b8c9d0"),
  "name": "Manish",
  "age": 23
}
```

Q: Can we create our own `_id`?

Answer:

> **Yes. We can provide our own unique value for `_id`, such as a string or number. If we don't provide one, MongoDB automatically generates an `ObjectId`.**

Example:

```
db.users.insertOne({
    _id: "USER001",
    name: "Manish"
});
```

One-Liner

> **`_id` is the unique identifier for every MongoDB document, similar to a primary key in SQL. By default, MongoDB generates it as an `ObjectId`, which is a 12-byte value containing a timestamp, a random value, and an incrementing counter to ensure uniqueness.**

Q: Can two documents have the same `_id`?

> **No. The `_id` field must be unique within a collection. If you try to insert another document with the same `_id`, MongoDB throws a duplicate key error.**

Q: Difference between `updateOne` and `updateMany`? A: `updateOne` updates only the first document matching the filter. `updateMany` updates all matching documents. Both use the same operators (`$set`, `$inc`, etc.).

what is CRUD?

CRUD stands for **Create, Read, Update, and Delete**. These are the four basic operations used to interact with a database.

CRUD Operation	Purpose	MongoDB Methods
Create	Add new documents to a collection	`insertOne()`, `insertMany()`
Read	Retrieve documents from a collection	`find()`, `findOne()`
Update	Modify existing documents	`updateOne()`, `updateMany()`
Delete	Remove documents from a collection	`deleteOne()`, `deleteMany()`

Question	MongoDB Query
Find all users	`db.users.find()`
Find one user	`db.users.findOne({name:"Manish"})`
Find users older than 18	`db.users.find({age:{$gt:18}})`
Find users between 20 and 30	`db.users.find({age:{$gte:20,$lte:30}})`
Find users from Delhi	`db.users.find({city:"Delhi"})`
Find users from Delhi or Mumbai	`db.users.find({city:{$in: ["Delhi","Mumbai"]}})`
Find users not from Delhi	`db.users.find({city:{$ne:"Delhi"}})`
Find users whose name starts with "M"	`db.users.find({name:/^M/})`
Sort users by age (Ascending)	`db.users.find().sort({age:1})`
Sort users by age (Descending)	`db.users.find().sort({age:-1})`
Get first 5 users	`db.users.find().limit(5)`
Skip first 10 users	`db.users.find().skip(10)`
Count all users	`db.users.countDocuments()`
Count users from Delhi	`db.users.countDocuments({city:"Delhi"})`
Show only name and email	`db.users.find({}, {name:1,email:1,_id:0})`

Question	Answer
What is CRUD?	CRUD stands for Create, Read, Update, and Delete—the four basic operations performed on a database.
Difference between `find()` and `findOne()`?	`find()` returns all matching documents, while `findOne()` returns only the first matching document.
Difference between `updateOne()` and `updateMany()`?	`updateOne()` updates the first matching document, while `updateMany()` updates all matching documents.
Difference between `deleteOne()` and `deleteMany()`?	`deleteOne()` deletes the first matching document, while `deleteMany()` deletes all matching documents.
What is Projection?	Projection is used to return only the required fields from a document, reducing unnecessary data transfer.